Project 2 on Neural networks

Tim Herrmann Holmen

FYS-STK4155
Department of Physics

This project investigates feed-forward neural networks for regression and classification. For regression, a network trained with stochastic gradient descent is used to approximate the Runge function and compared with ordinary least squares, Ridge, and Lasso regression. For classification, a neural network is trained to recognise handwritten digits from the MNIST dataset. Backpropagation gradients are checked against automatic differentiation, and model parameters are selected using cross-validation, using scikit learns GridSearchCV. The selected regression network achieves impressive test performance, although linear models offer shorter training times. For the regression problem, the selected neural network achieves a CV MSE of 0.01846 ± 0.00135 and a test MSE of 0.01434. Compared with 8 degree ridge, rasso and linear regression with respectively $0.01477, 0.01478, 0.01495$ in test MSE. The neural network had the longest mean fit time with $2593ms$ and linear regression the shortest with $3ms$, a substantial difference. For digit classification, the selected model achieves 84.4% cross-validation accuracy and 89.22% accuracy on the test set. The results illustrate how neural-network performance depends on model architecture, optimisation, the data and the task being considered.

I. INTRODUCTION

A. Motivation

Aritficial neural networks have shown great results and improvements in the last years, impacting fields such as science, poltics and philosophy. Large language models, image detection, drones and autonomous systems, all have variations of artificial neural networks as its basis. In politics we can hear about intelligent machines, aritificial intelligence and super intelligence. The motivation for this project is to learn about neural networks, demistify its supernatural appearance, and understand the mechanics and its capacity to learn and solve known functions and data relations. The method and results will be compared to known methods of linear regression and classification.

B. What i have done

1. Regression problem

We have created a feed forward neural network; FFNN, used stochastic gradient descent; SGD, to find the minimum of the cost function, which is the MSE for the regression problem. The gradients are calculated using backpropagation, which uses the kernal rule of derivation on the FFNN in reverse; from the output and target MSE derivative to the derivative of the parameters of the first layer. The gradients are compared with jax, automatic differentiation, to ensure agreement. In the SGD the descent method used on the parameters and gradients is the Plain gradient descent; regular updating, RMSprop; complex updating, Adam; best updating. Different layer width and depths are explored, along with the activation functions in the hidden layers, different learning rates are explored and compared. The output layer has the identity as activaiton function. The choice of activation function is the identity since the problem of choice is a regression problem. This network is compared with OLS for the one dimensional runge function. Further L1 and L2 penalties are applied to the cost function, with a variable penalty value λ, and these models are compared with Lasso and Ridge.

2. Classification problem

For classification, the network uses the softmax function in its output layer and cross-entropy as its cost function. These choices, together with the dataset, distinguish this network from the regression network. The task is multiclass classification of handwritten digits from MNIST; see https://www.tensorflow.org/datasets/catalog/mnist. We investigate the hidden-layer width, learning rate γ, and the plain, RMSprop, and Adam update methods. We also apply L1 and L2 penalties with different values of the regularisation parameter λ.

C. In this report

The theory section II, explains the cost functions and their gradients, and the activation functions with their gradients. Which are used in this project. The method section III, explains the data used in this project, along with the neccesary alghoritms for creating a neural network, with emphasis on the backpropagation alghorithm used for creating the gradients neccesary for training the network, for both the regression problem and the classification ppoblem. For training stochastic gradient descent

is explained briefly. The neuralnetregressor class is explained along with the implemenation, model selection and analysis. The neuralnet classifier and its data are explained in the last section of the methods, along with its implementation, model selection and analysis. The results section IV, shows the results of the regression and classification network implementation, the model selection and analysis of the results using tables, figures and heatmaps. The V discusses the results, the implementation, and the models used along with its parameters. The VI rounds up everything discussed and explored in this project.

The full project can be found at https://github.com/Timholmen96/FYS-STK4155/tree/467585f0bfdbf69ab54de5daa74d59e5d528582a/Project2/Project-Code

II. THEORY

A. Cost functions and their gradients

The loss function is the most important function for training, since training a model means to minimize the cost function of the model. To minimize the cost function, we use backpropagation to find the gradient of the cost, with regards to and gradient descent.

1. MSE

When we have our calculated output value $\tilde{y}$ and the true output value y, with n data points, the mean-squared error and its derivative is

$$MSE(y,\tilde{y}) = \frac{1}{n}\sum_{i=0}^{n-1}(y-\tilde{y})^2 \quad \frac{1}{n}||y - X\Theta||_2^2 \tag{1}$$

a. L2 norm which corresponds to the Ridge cost function.

$$MSE + \lambda||\Theta||_2^2 \quad \Theta(W,b) \tag{2}$$

b. L1 norm which corresponds to the Lasso cost function.

$$MSE + \lambda||\Theta|| \quad \Theta(W,b) \tag{3}$$

(Hjorth-Jensen, 2016, p. 86-91)

2. Binary cross entropy: log loss

With two classes, we assume a binary output $y \in \{0,1\}$, and a predicted probability $p \in [0,1]$ which is the output of the sigmoid function of z_i. Where $z_i = x_i^T\Theta$ is the linear combination of the input features and the model parameters Θ. For a single example, data point i, the binary cross entropy cost function is defined as

$$C(\Theta) = -\sum_{i=1}^{n}(y_i \log p_i + (1-y_i)\log[1-p_i]) \tag{4}$$

$$p_i = \sigma(z_i) = \frac{1}{1+\exp(-z_i)} \tag{5}$$

$$\tag{6}$$

The cost function is derived from the maximum likelihood estimation of the Bernoulli distribution, the negative log likelihood of the Bernoulli distribution. This is known as the cross entropy, and it is the loss function used to train binary classification models. With the linear function $z_i = x_i^T\Theta$, the gradient of the cost function with respect to the model parameters Θ is given by

$$\frac{dC(\theta)}{d\theta} = -X^T(y-p) \tag{7}$$

For N data points, the mean binary cross entropy is defines as

$$\frac{1}{N}\sum_{i=1}^{N}C(\theta)_i$$

And the term $1/N$ follows through te calculations to give the mean binary cross entropy gradient, which is given by

$$\nabla_\Theta C(\Theta) = -\frac{1}{n}X^T(y-p) \tag{8}$$

The cross entropy L1 and L2 norm is supplemented like the ordinary MSE.

3. Multiclass cross entropy cost function

For k klasses, one uses the soft max function to calculate the probability of each class. Again we have the linear function $z_k = X^T\Theta_k$ for each class, over all features and data points, the softmax functions is defined as

$$p(C=k \mid x) = \frac{\exp(z_k)}{\sum_{l=0}^{K-1}\exp(z_l)} k = 0,\ldots,K-1 \tag{9}$$

The nominator is for class k, and the denominator is the sum of all classes. The multiclass cross entropy cost function is defined as

$$C(\Theta) = -\sum_{i=1}^{n}\sum_{k=0}^{K-1} y_{ik}\log p(C=k|x_i) \tag{10}$$

Which is summed over all data points and classes. The gradient of the multiclass cross entropy cost function is given by

$$\frac{dC}{d\Theta_k} = -X^T(y_k - p_k) \tag{11}$$

The cross entropy L1 and L2 norm is supplemented like the ordinary MSE. Stability: The softmax function can be numerically unstable for large values of z_k, for a large value the e exponential function can overflow. Since the softmax is invariant udner subtracting a constant frome every score, one subtracts the largest before exponentiating:

$$p_k = \frac{\exp(z_k - z_{max})}{\sum_l \exp(z_l - z_{max})}$$

, $z_k = x^T \Theta_k$. (Hjorth-Jensen, 2016, p. 151-156)

B. Activation functions and their gradients

1. Sigmoid logit function

$$\sigma(z) = \frac{1}{1+\exp(-z)} \tag{12}$$

$$\sigma'(z) = \sigma(z)(1-\sigma(z)) \tag{13}$$

2. RELU function

$$ReLU(z) = \max(0, z) \tag{14}$$

$$ReLU'(z) = \begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases} \tag{15}$$

(Hjorth-Jensen, 2016, p. 214-242)

III. METHODS

A. The Data: Regression

In this project, the input data is normally distributed, and consist of 100 data points. The target data is generated by Runge's function, with added noise, Runge Function which is a one-dimensional function of second order.

$$f(x) = \frac{1}{1+25x^2}$$

The noise is normally distributed with a mean of 0 and a standard deviation of 0.1.

1. Reshaping the data

The input and output data are reshaped into column vectors, (100,1), this is useful since most machine learning tools expect input features as a 2D array. For linear regression problems, the input data is expressed in terms of polynomial features, which is done by creating the design matrix of variable degree with the input data x, and adjusting the features during training, this is equal for ordinary least squares, lasso and ridge. For the neural network this is not necessary, since the network architecture, the number of nodes and layers, replaces the polynomial features, and the weights and biases are adjusted during training.

B. The algorithm

The set-up.

1. Fix the architecture: the number of inputs and outputs, the number of hidden layers and the number of nodes in each.
2. Choose activation functions for the hidden layers and for the output layer, the latter dictated by the range of the target.
3. Choose the cost function, again dictated by the distribution of the target, and any regularisation terms with their hyperparameters
4. Choose the optimiser - plain gradient dsecent, RMSprop, adam - and its learning rate.
5. Initialise the weights and biases

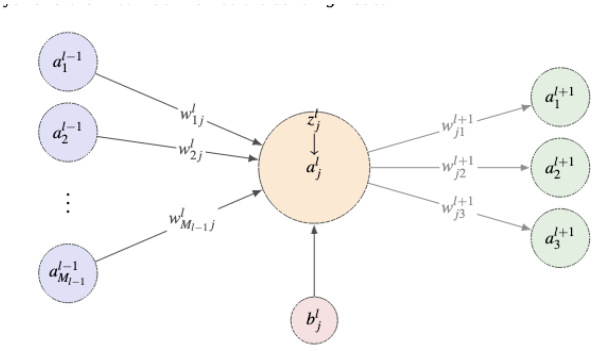


FIG. 1 The index convention for the equations below, seen from a single node $a_j^{(l)}$. The node j in layer l collects the outputs $a_i^{(l-1)}$ of all M_{l-1} nodes in the previous layer, weighted by $w_{ij}^{(l)}$, adds its own bias $b_j^{(l)}$ to form the pre-activation $z_j^{(l)}$, and applies its activation function σ to obtain $a_j^{(l)}$, which it then sends on to every node of layer $l+1$ through the weights $w_{jk}^{(l+1)}$.

For a single node, the pre-activation and activation are

$$z_j^{(l)} = \sum_{i=1}^{M_{l-1}} w_{ij}^{(l)} a_i^{(l-1)} + b_j^{(l)},$$

$$a_j^{(l)} = \sigma\left(z_j^{(l)}\right).$$

(Hjorth-Jensen, 2016, p. 326–328)

Each iteration:

1. *Forward pass.* Set $a^{(0)} = x$ and compute $z^{(l)}$ and $a^{(l)} = \sigma(z^{(l)})$ for $l = 1, 2, \ldots, L$, storing every $z^{(l)}$ and $a^{(l)}$:

   $$z^{(l)} = a^{(l-1)} W^{(l)} + b^{(l)}, \qquad a^{(l)} = \sigma\left(z^{(l)}\right).$$

2. *Output error.* For a differentiable output activation, compute

   $$\delta^{(L)} = \sigma'\left(z^{(L)}\right) \odot \frac{\partial C}{\partial a^{(L)}}.$$

   For softmax with cross-entropy, use

   $$\delta^{(L)} = \frac{a^{(L)} - y}{n}.$$

3. *Backward pass.* For $l = L-1, L-2, \ldots, 1$, compute

   $$\delta^{(l)} = \left(\delta^{(l+1)} (W^{(l+1)})^\top\right) \odot \sigma'\left(z^{(l)}\right).$$

4. *Gradients.* Form $\frac{\partial C}{\partial W^{(l)}} = (a^{(l-1)})^\top \delta^{(l)}$ and $\frac{\partial C}{\partial b^{(l)}} = \sum_{i=1}^{n} \delta_i^{(l)}$.

5. *Update.* With stochastic gradient descent and learning rate γ, update the weights and biases:

$$W^{(l)} \leftarrow W^{(l)} - \gamma (a^{(l-1)})^\top \delta^{(l)}, \qquad b^{(l)} \leftarrow b^{(l)} - \gamma \sum_{i=1}^{n} \delta_i^{(l)}.$$

(Hjorth-Jensen, 2016, p. 350)

1. Backpropagation

The backpropagation is the most important part of the neural network, as it is this process that creates the gradients of each layer, which are used to update the weights and biases. This process is different depending on the cost function, since the derivative of the cost function is the value we want to minimize. Thus it is different for the regression problem, which uses mean squared error, and the classification which uses cross entropy. The input layers, the z values and the output values a is generated by the feed forward pass saver, the backpropagation iterates backwards, from the last layer to the first. Regressor, mean squared error:

$$\text{Output layer:} \quad \frac{\partial C}{\partial a^{(L)}} = \text{cost function derivative}, \tag{16}$$

$$\text{Hidden layer:} \quad \frac{\partial C}{\partial a^{(l)}} = \delta^{(l+1)} \left(W^{(l+1)}\right)^\top, \tag{17}$$

$$\delta^{(l)} = \frac{\partial C}{\partial z^{(l)}} = \frac{\partial C}{\partial a^{(l)}} \odot \sigma'\left(z^{(l)}\right), \tag{18}$$

$$\frac{\partial C}{\partial W^{(l)}} = \left(a^{(l-1)}\right)^\top \delta^{(l)}, \tag{19}$$

$$\frac{\partial C}{\partial b^{(l)}} = \sum_{i=1}^{n} \delta_i^{(l)}, \tag{20}$$

$$\nabla^{(l)} = \left(\frac{\partial C}{\partial W^{(l)}}, \frac{\partial C}{\partial b^{(l)}}\right). \tag{21}$$

Cross entropy, classification.

$$\text{Output layer:} \quad \Delta^{(L)} = \frac{\partial C}{\partial Z^{(L)}} = \frac{A^{(L)} - Y}{n}, \tag{22}$$

$$\text{Hidden layers:} \quad \begin{aligned} \Delta^{(l)} &= \frac{\partial C}{\partial Z^{(l)}} \\ &= \left(\Delta^{(l+1)} (W^{(l+1)})^\top\right) \\ &\quad \odot \sigma'\left(Z^{(l)}\right), \\ &\quad l = L-1, \ldots, 1. \end{aligned} \tag{23}$$

C. Stochastic Gradient Descent

The stochastic gradient descent function trains the neural network over a specified number of epochs. One epoch is one pass through the training data. At the start of each epoch, the data are shuffled and divided into mini-batches, each containing up to `batch_size` examples. Thus, for N training examples and batch size B, each epoch contains $\lceil N/B \rceil$ mini-batches. The network is updated after each mini-batch, using gradients calculated from that batch with backpropagfation. This is called mini-batch stochastic gradient descent. In the implementation, the backpropagation alghorithm calculates the mini-batch gradients, and the selected optimizer, plain, adam or rmsprop, updates the weights and biases. L1 or L2 penalty terms can also be added to the weight gradients, corresponding to Lasso and Ridge. The optimizer is selected separately from the mini-batch training procedure.

D. Neuralnetregressor

This estimator wraps the network in a scikit-learn-compatible interface. Its `fit` method creates the network

using the requested input size and hidden-layer architecture. It selects the hidden activation function and its derivative, uses the identity function at the output layer, and supplies the mean squared error cost and its derivative.

Training is delegated to `train_sgd` or `train_plain`, according to the `descent` parameter. For SGD, the `method` parameter selects plain gradient descent, Adam, or RMSprop. The training function updates the weights and biases; the fitted network and training history are retained by the regressor.

The `predict` method passes new inputs through the fitted network and returns a one-dimensional NumPy array with one continuous prediction per example.

E. Implementation and analysis of Regressor

1. Model verification

The neuralnetregressor is implemented as an object, and verified against manually creating a neural network object and using stochastic gradient descent to train the model, both objects predict on the same data and the mean squared error are calculated on the same data, to ensure they agree. The gradients calculated by the backpropagation algorithm, are valified by calculating the gradient on the same data by automatic differentiation using jax. Jax is a verified solution used by scientists and data analysists, thus this verification ensures that the backpropagation calculates the correct gradients.

a. Gradient and training validation We tested a network with one hidden layer of 50 nodes and a network with two hidden layers of 50 and 100 nodes. Both used sigmoid hidden-layer activations. For all three training methods, the manually calculated gradients matched those from `jax.grad` according to `np.allclose`. The regressor and manual SGD implementations also produced matching predictions.

TABLE II Gradient validation and SGD training MSE for a network with two hidden layers containing 50 and 100 nodes, respectively.

Method	Gradient val.	Regressor val.	SGD train MSE
Plain	Passed	Passed	0.1084
Adam	Passed	Passed	0.01873
RMSprop	Passed	Passed	0.01394

TABLE I Gradient validation and training MSE for a network with one hidden layer of 50 nodes.

Method	Gradient val.	Regressor val.	SGD train MSE
Plain	Passed	Passed	0.1031
Adam	Passed	Passed	0.02404
RMSprop	Passed	Passed	0.02142

The gradient validation passed for all methods, and both the regressor and the manual use of sgd agreed. Adam and RMSprop achieved lower training MSE than plain stochastic gradient descent. The code can be found at https://github.com/Timholmen96/FYS-STK4155/blob/467585f0bfdbf69ab54de5daa74d59e5d528582a/Project2/Project-Code/Project2_Test_Regressor.ipynb

F. Model selection

1. Skicit Learn

Scikit-learn has multiple classes that can be used to validate and select between different models. The following classes have been utilised in the analysis of the Neuralnetwork regressor,

1. **KFold**: Creates k folds with shuffled values, k is set to 5 in this project. With random state = 2026.

2. **make$_\mathbf{p}$ipeline**: Creates a pipeline with scikit's StandardScaler() from the training data, with the NeuralNetRegressor as a chosen model.

3. **cross$_\mathbf{v}$al$_\mathbf{s}$core**: Splits the input data and target values, using the Kfold splitter, fits the pipeline on the training portion of each split, and evaluates it on the validation set of the split. Returns one score per split, with standard deviation.

4. **cross$_\mathbf{v}$alidate**: Does the same operation as cross val score, returns the training score in addition to the validation score for each fold, with standard deviation. Both classes should return the same validation score.

5. **GridSearchCV**: The GridSearchCV is the most extensively used class in this project. It uses the same Kfold class and pipeline as the previous cross validation classes. It has an additional $parameter_grid$ input, which uses the input values of the neuralnetregressor as variable model parameters, this allows for models selection on different parameters, such as the learning rate, number of hidden layers and gradient descent methods, plain, adam and rmsprop. This class creates and stores models for all chosen parameters within the grid, and fits on the chosen data using the regressor, and returns the cross validation scores on the training

and validation set, the chosen score for the regressor is the negative mean squared error.

2. Cross Validation

The first **make$_p$ipeline** is created with the NeuralNetRegressor class with one hidden layer of 50 nodes, sigmoid as activaiton function of the hidden layer, a learning rate of 0.01 with plain gradient descent as the standard method, the epoch is set to 100 with stochastic gradient descent with a batch size of 10. The seed is set to 2026. **KFold** is set with 5 folds, shuffle and random state with seed 2026. Both cross validation methods on this particular regressor model, is chosen with negative mean squared error as the scoring value.

3. GridSearchCV

The gridsearch cross validation is done using the chosen Kfold and neural network regression class as input variables. The cross validation is done with a different parameter grid and GridSearchCV model for each step, after each step the best model parameters within one standard error of the mean squared error is chosen, and used in the next neural regressor model, which in turn is the chosen model for the new GridSearchCV model. GridSearchCV stores the best model and its scorer, the best model corresponds to the lowest validation mean squared error of all the models. The first parameter grid is. Best model parameters within ONE-SE is found

TABLE III Parameter grid 1

Parameter	Values				
Gamma	0.001	0.003	0.01	0.03	0.01
Method	rmsprop	adam	plain		
Hidden	(50,)	(50, 100)			

through analysing the results of the GridSearchCV, the best model parameters gamma $-learningrate$, method and hidden layers are chosen for the next neuralnetregressor model input of the next GridSearch. The second parameter grid is. Here lambda equals the value of the

TABLE IV Parameter grid 2

Parameter	Values			
Activation	sigmoid	relu		
Hidden	(50,)	(30,)	(50, 100,)	(50, 100, 100,)

TABLE V Parameter grid 3

Parameter	Values			
Penalty	None	L1	L2	
Lambda	0.0001	0.001	0.01	0.1
Gamma	0.003	0.01	0.03	0.1

penalty, L1 and L2 are the norms. The last parameter grid is split in half, where lambda is set to zero when the penalty is None, else the lambda values are as shown in the table.

a. One SE It is important to specify what is meant by a one se model in this project. When a model is chosen during cross validation, based on the lowest mean squared error, if another model has a mse is within one standard error of the lowest, the choice between these two models would be arbitrary if mse was the only qualification, since one cannot tell, which model actually has the lowest mse, because the standard error reaches around both models mse. Thus, between these two models the least complex, most efficient model is chosen, this is also true if more than two models are within one se of the best. What model to choose then within one se of the best model, is then dependent on the model and the parameters which are varied during the cross validation.

4. Linear Regression

The first parameter grid is compared with linear regression, imported from skicit learn, the pipeline includes $StandardScaler()$ and $PolynomialFeatures()$. The pipeline is cross validated over polynomial degrees $(1, 16)$ using GridSearchCV.

5. Dissapearing gradient

The second parameter grid explore if there is signs of dissapearing gradients using the sigmoid activation function, as opposed to the relu function, with a varied network layer depth and width.

6. Lasso and Ridge

The second parameter grid, which explores widderent penalty norms and values, along with learning rate during the cross validation search. The results are then compared with Lasso and Ridge of degrees $(1, 16)$ and different penalty values, α, using Sci-kit Linear model classes, pipeline and GridSearchCV. The Ridge linear model uses singular valua decompostion "svd" as its solver, while Lasso has a max iteration count of 100000.

7. Final test

The best model from the OLS comparison is tested on data generated by a different $seed = 1$, the same is done for the Lasso and Ridge comparison. The data is predicted on the test data and the mean squared error

is calculated between the predicted values and the target test values.

G. Neuralnetclassifier

The `NeuralNetClassifier` class provides a scikit-learn-compatible interface for training the project's neural network on classification tasks. During `fit`, it records the unique class labels and converts the training targets to one-hot vectors. The network has one output node per class, with softmax producing a probability distribution over the classes. Hidden layers use the selected activation function, and training minimises the cross-entropy loss.

The class delegates mini-batch training to `train_sgd`. Its parameters specify the hidden-layer sizes and activation, learning rate, optimiser, regularisation, number of epochs, batch size, and random seed. The `predict_proba` method returns the predicted class probabilities, while `predict` selects the class with the highest probability and maps it back to its original label. Its `accuracy` method evaluates predicted labels against the targets. This interface lets the classifier be used with scikit-learn tools such as pipelines and cross-validation. The activation function of the output layer is the softmax, and the cost function is the cross entropy, as mentioned earlier.

1. The data

The data used in the classification problem is the multiclass MNIST data set of handritten digits, see the description of the data set at https://www.tensorflow.org/datasets/catalog/mnist. The data set comes with a canonical split into 60 000 training images and 10 000 test images, each a $28x28$ array of grey scale pixel values between 0 and 255. The training data is separated randomly into two smalles subsets with 1000 data points each, as shown in the code section below: The network

```
import numpy as np
data = np.load("mnist.npz")
x_train = data["x_train"].reshape(len(data["x_train"]), -1) / 255.0
y_train = data["y_train"]
rng = np.random.default_rng(2026)
# 1000 data points in the subset
sub = np.concatenate([rng.choice(np.where(y_train == c)[0], 100, replace=False) for c in range(10)])
X_sub, y_sub = x_train[sub], y_train[sub]
# Don't touch this!
x_test_full = data["x_test"].reshape(len(data["x_test"]), -1) / 255.0
y_test_full = data["y_test"]
```

FIG. 2 The canonical split of the data set and the subset of the training data, flattening and scaling of input data

takes a vector as input, so the image is flattened to a vector of 784 features and scaled to the interval $[0, 1]$. (Hjorth-Jensen, 2026). The full test data is kept aside for the last test, so that no data leaks into the training.

2. GridSearchCV

The pipeline for the classification problem is established with Scikit-learns NeuralNetClassifier with 100 epochs and a batch size of 50, with 1000 data points this corresponds to 20 mini batches per epoch. The Kfold is created using 5 folds with random state 2026, using the Scikit class StratifiedKFold, a variation of the KFold that returns stratified folds, the folds are made by preserving the percentage of samples for each class in the target data in a multiclass classification setting. (Str, 2026) The parameter grid is created in two parts, with a common grid consisting of parameter values:

$$hidden = [(32,), (64,), (128,), (64, 128,)]$$

$$gamma = [0.001, 0.01, 0.03, 0.1]$$

$$method = [\text{"}plain\text{"}, \text{"}adam\text{"}, \text{"}rmsprop\text{"}]$$

$$activation = [\text{"}relu\text{"}, \text{"}simoid\text{"}]$$

The full parameter grid consist of the common grid without Penalty and with L1 and L2 Penalty with penalty values:

$$lambda = [0.0001, 0.001, 0.01]$$

The Gridsearch is created with these parameter values, the Stratified Kfold, the NeuralNetClassification class with standard values and scoring = "accuracy". The best model is chosen by CV validation accuracy, using the output of the GridSearch. The highest accuracy corresponds to the best model. If there exist some models in the One SE threshold of the best, the most simple and accurate network is chosen. The best model is chosen from the GridSearchCV is fitted on the full test data, and the accuracy is calculated using scikit learn `accuracy_score` with the predictions and the target data, each correct classification is summed up and divided by the total number of data points, which equals 10000.

3. Confusion matrix

The confusion matrice uses the predicted values and the true target values, to create a $n_{classes}xn_{classes}$ matrix, with 10 classes corresponding to the numbers $[0, 9]$ the result is a $10x10$ matrix where each point in the matrice corresponds to one of four possibilites, true positive (tp); false negative (fn), false positive (fp), true negative (tn). The confusion matrix evaluates the accuracy of a classification, where the goal is to achieve true positive values, which are the values along the diagonal of the matrix and corresponds to a correct claccification; the true

classification corresponds to the predicted classificaiton, and both are positive that the classification is the same number. False negative corresponds to a wrong classification, the true value is positive for the particular number, but the predicted value is negative, and classifies another number. False positive is when the true value is negative for the particular number, and the predicted value is positive, and classifies the particular number wrongly. True negative corresponds to a correct negative classification, both the target and the predicted classification classifies the number negatively, it is not this particular number. A positive classification is given the number 1, and a negative classification corresponds to 0, true and false. When summing over all data points, each point in the matrix is added 1, diagonally for true positive, and off diaogonally for the false negative and false positive, false negatives are horisontal, and false positive are vertical along the matrix. This matrix gives a visual representation of the accuracy of the chosen model. (Con, 2026)

IV. RESULTS

A. NeuralNetRegressor

1. First CV GridSearch

	Learning rate	Method	Hidden layers	Mean CV MSE	Std CV MSE	Mean fit time (s)
0	0.100	adam	(50, 100)	0.017822	0.002011	4.247238
1	0.030	adam	(50, 100)	0.017885	0.002712	4.194257
2	0.010	adam	(50, 100)	0.018065	0.002770	4.197701
3	0.010	adam	(50,)	0.018463	0.003024	2.396347
4	0.003	adam	(50, 100)	0.019957	0.003203	4.287275
5	0.030	rmsprop	(50, 100)	0.022913	0.006340	3.950180
6	0.030	adam	(50,)	0.023416	0.010491	2.377712
7	0.010	rmsprop	(50, 100)	0.024179	0.009194	4.621853
8	0.003	rmsprop	(50,)	0.027679	0.006000	2.458915
9	0.003	adam	(50,)	0.028000	0.006111	2.378305
10	0.100	adam	(50,)	0.028687	0.006502	2.340376

FIG. 3 **FirstCVGridSearch** : First 10 model scores with standard error and mean fit time.

The first GridSearchCV results are plotted in a heatmap. As seen from the heatmap, the best model

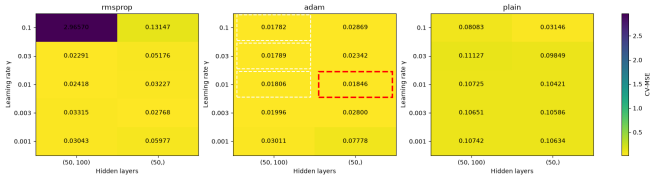


FIG. 4 **Heatmap** for methods rmsprop, adam and plain, showing learning rate and hidden layers as axis values, with the color bar showing cross validation MSE. The dotted lines show all models within one standard error of the best, and the red dotted line shows the chosen model, which is the model with least hidden layers and nodes

within one se has one hidden layer with 50 nodes, adam as gradient descent method and a learning rate of 0.01. The model with the lowest CV MSE has two hidden layers, with 150 nodes in total, since four models are within One-SE of the lowest, the model with least nodes is chosen.

2. OLS comparison and test set

The chosen model from the first gridsearch cross validation is compared to a linear regression model, where the linear regression model is crossvalidated over chosen degrees with Gridsearch, the lowest CV MSE models and the chosen ONE-SE models are shown below, along with their fit time and number of parameters. The neuralnet-

Model	Architecture	CV MSE ± SE	Parameters	Fit time (ms)
Network	gamma=0.1, hidden=(50, 100), method=adam	0.01782 ± 0.00101	5301	4392.9
OLS	degree=11	0.02111 ± 0.00324	12	3.7

Model	One-SE selection	CV MSE ± SE	Parameters	Fit time (ms)
Network	hidden=(50,), method=adam, gamma=0.01	0.01846 ± 0.00151	151	2414.2
OLS	degree=8	0.02253 ± 0.00172	9	4.1

FIG. 5 The best model and the One SE selected model for both the neural network and the linear regression models, OLS. Including the cross validated parameter values in the architecture, number of parameters in the model and the fit time in ms

work achieves a better CV MSE than ols, but the number of parameters and the average fit time is substantially larger. Strangely enough the 8 degree polynomial has a longer fit time than the 11 degree polynomial fit. The average fit time for the neural network is halved for the one-se model, the choice pays off in data cost. Both one-se models are then fitted to test data and the test mse is calculated from the target data. The best test mse is the

		Test MSE
Model	Selection	
Network	Lowest CV-MSE	0.01235
OLS	Lowest CV-MSE	0.01326
Network	One-SE	0.01434
OLS	One-SE	0.01495

FIG. 6 Test MSE for the lowest mse model and the chosen one se model

neural network with two hidden layers, which beats both ols models. Of the One SE models the network beats OLS, but the 11 degree ols has a lower mse than the one-se network. Still the One-SE network parameters are

chosen for further crossvalidation. The parameters are: *hidden* $= (50,)$, learning rate $\gamma = 0.01$ and method is adam.

3. Second CV Gridsearch

The next Gridsearch is cross validated for depth, width and activation fuctions for the hidden layers.

	Activation	Hidden layers	Mean CV MSE	CV SE	Mean fit time (s)	ONE-SE
0	relu	(50, 100, 100)	0.017265	0.001297	4.134905	True
1	sigmoid	(50, 100)	0.018065	0.001239	4.318662	True
2	relu	(50, 100)	0.018123	0.001531	4.017302	True
3	sigmoid	(50,)	0.018463	0.001352	2.818784	True
4	sigmoid	(30,)	0.019372	0.001533	4.625208	False
5	sigmoid	(50, 100, 100)	0.020060	0.002277	5.956798	False
6	relu	(50,)	0.025431	0.001909	2.664996	False
7	relu	(30,)	0.026320	0.002636	2.230363	False

FIG. 7 **SecondCVGridSearch** : All model scores with standard error and mean fit time, along with a One SE validation for each model, with the lowest CV MSE as reference

From this figure we see that there are four models which are witin One-SE of the lowest CV-MSE, interestingly with a varied choice of activation functions and hidden layers. By choosing for the least amount of nodes, the choise is one hidden layer with 50 nodes and sigmoid as activation function.

4. Last CV Gridsearch

	Penalty	Lambda	Gamma	Mean CV MSE	CV SE	Mean fit time (s)	Within One-SE
0	NaN	0.0000	0.010	0.018463	0.001352	2.593421	True
1	l1	0.0001	0.010	0.018959	0.001335	2.877847	True
2	l2	0.0001	0.010	0.022432	0.001519	2.819707	False
3	l1	0.0001	0.030	0.022503	0.004114	2.868898	False
4	l1	0.0010	0.010	0.023399	0.001863	2.876308	False
5	NaN	0.0000	0.030	0.023416	0.004692	2.959481	False
6	l1	0.0010	0.030	0.023777	0.002970	2.746234	False
7	l1	0.0010	0.100	0.024103	0.003814	3.164458	False
8	l2	0.0001	0.100	0.024991	0.004321	2.838009	False
9	l2	0.0001	0.030	0.025653	0.003132	2.847799	False
10	l1	0.0001	0.100	0.025809	0.004695	2.889132	False

FIG. 8 **ThirdCVGridSearch** : The 10 first model scores with standard error and mean fit time, along with a One SE validation for each model, with the lowest CV MSE as reference

In this last cross validation, the One-SE chosen model is also the model with the lowest MSE. The L1 penalty model with a penalty value of 0.0001 is within One-SE of the lowest, which corresponds to Lasso. Both models have the same learning rate 0.01. The best model is chosen on basis of the mean fit time in seconds, where the best model have no penalty. To sum up the chosen parameters over all three GridSearch cross validation results are:

TABLE VI Selected neural network parameters after the three grid-search stages.

Parameter	Selected value
Method	Adam
Learning rate γ	0.01
Hidden-layer architecture	$(50,)$
Activation function	Sigmoid
Penalty	None

$$CVMSE = 0.01846 \pm 0.00135$$

5. Ridge and Lasso

```
Ridge candidates within One-SE:
  degree=8, lambda=0.0001, CV-MSE=0.02241 +/- 0.00172
  degree=10, lambda=0.0001, CV-MSE=0.02242 +/- 0.00145
  degree=11, lambda=0.0001, CV-MSE=0.02103 +/- 0.00207
  degree=12, lambda=0.0001, CV-MSE=0.02168 +/- 0.00247

Lasso candidates within One-SE:
  degree=8, alpha=0.0001, CV-MSE=0.02228 +/- 0.00172
  degree=12, alpha=0.0001, CV-MSE=0.02241 +/- 0.00111
  degree=13, alpha=0.0001, CV-MSE=0.02154 +/- 0.00169
  degree=14, alpha=0.0001, CV-MSE=0.02138 +/- 0.00194
  degree=15, alpha=0.0001, CV-MSE=0.02138 +/- 0.00215
```

FIG. 9 **RidgeandLasso** : All models within one One SE, with the lowest CV MSE as reference

The chosen degrees are thus the same as for linear regression, 8 degree polynomial, which corresponds to 9 parameters with the intercept.

6. Final Test

The best models are fitted on the same full data set with a different *seed* $= 1$, and mse is calculated from the traget values. The neural network achieves the best test

Model	Test MSE	Mean fit time (s)
NN (One-SE)	0.014336	2.593421
Ridge (One-SE)	0.014772	0.003050
Lasso (One-SE)	0.014783	0.071396
OLS (One-SE)	0.014948	0.003040

FIG. 10 **Testset** : All chosen One SE models evaluated on the test set, once

MSE, although the fit time is substantially larger than the linear models. If the fit time were taken into account,

and the fact that the difference in mse is of order $1e-4$, the best model is either linear regression or Ridge, on the basis of both fit time and test mse.

B. NeuralNetClassifier

GridsearchCV implemenation on the NeuralNetclassifier produced several heatmaps, only the heatmaps including the best CV MSE and the One-SE CV Mse results are shown below. The full result can be found at: https://github.com/Timholmen96/FYS-STK4155/blob/467585f0bfdbf69ab54de5daa74d59e5d528582a/Project2/Project-Code/Project2_Classification_Init.ipynb

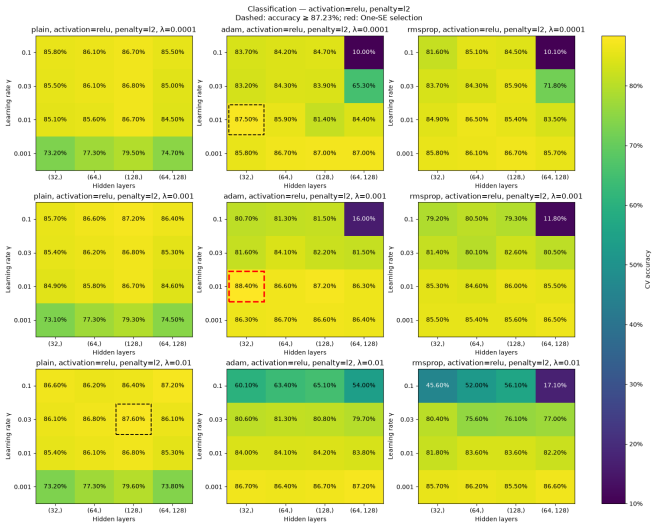


FIG. 11 **Heatmap** : Containing the best model from cross validation, in red dotted lines, and two One-SE models in black

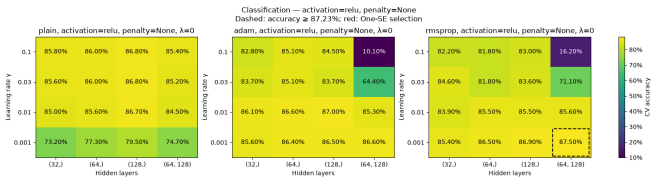


FIG. 12 **Heatmap** : Containing one One-SE model

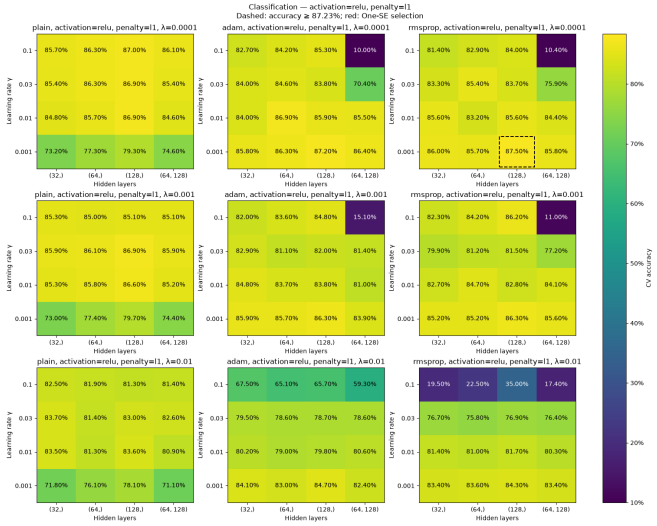


FIG. 13 **Heatmap** : Containing two One-SE models

From Fig $11-13$ we see that the model with the lowest cv mse is also the model within one standard error with the least parameters, one hidden layer of 32 nodes. The activation function in the hidden layer is `ReLU`, with L2 penalty with value $\lambda = 0.001$, the learning rate is $\gamma = 0.01$ and the gradient descent method is adam. The CV accuracy is 84.4% with standard error 1.17% which corresponds to $CV accuracy = 84.4\% \pm 1.17\%$.

1. Test Set once

The best model, chosen from the GridSearchCV, is applied to the separate test data and the accuracy is calculated with the target data, along with a confusion matrix for visual represenation.

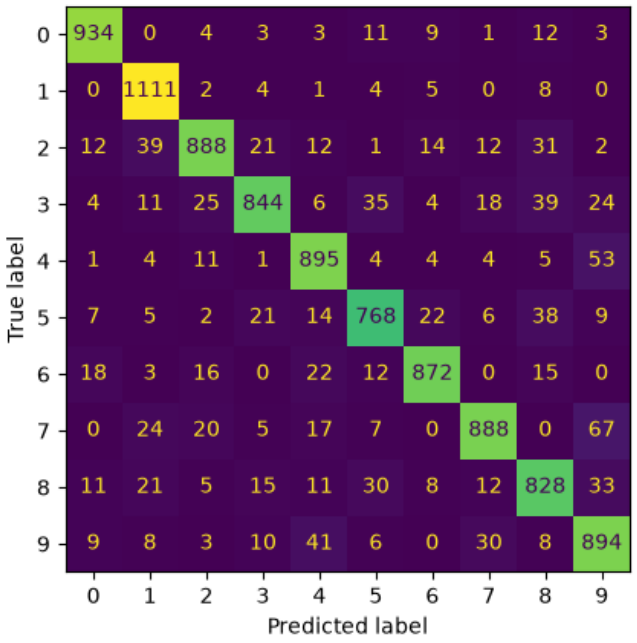


FIG. 14 **ConfusionMatrix** : Best model applied on separate test data

The overall accuracy is 89.22% The model has the highest accuracy with the digit 1 and lowest accuracy with the digit 5.

V. DISCUSSION

A. NeuralNetRegressor

1. GridSearchCV

One problem with the implementation of the GridSearchCV was the step wise implementation, the first implmentation established the best method, the second implementation established the best activation function and hidden layers, and the third established the penalty and the learning rate. If all these values were tested in the same grid search, some other model parameter values may have been proven best, as they vary according to the other model parameters. This information was lost in the step wise implementation. What was gained in this implementation is increased control, data visualisation and shorter fit time per model, which made the implementation more flexible.

2. Vanishing gradients

The problem of vanishing gradients, when using sigmoid in the hidden layers in deeper networks was not explored sufficiently. There are two reasons for this, the first is that the code updating the gradients, does not include a method to store the gradients, such that one could plot the evolving gradients, and compare them between the different layers. One knows that it is the layers that are furthest from the output layer that are most affected by this, so an L2 norm of the gradients in each layer could express, if the gradients were stored. (Kashyap, 2024) Also this project did not really explore networks with deep layers as the deepest network included three hidden layers, this could be explored in a seperate section by looking at the gradients of the different layers, of a deeper network, with sigmoid as activation function. The chosen network in this project, based one the lowest one SE CV MSE, had one hidden layer, and was not affected by this problem as strongly as a deeper network would.

3. Data cost

The final test concluded that the neural network had the lowest test MSE, and by this criterion it was also the best model. When looking at the fit time, the neural network does not compare with the linear models, and the small difference in test mse does not pay for the large difference in data cost. This problem could be solved by code optimization, but was not a part of this project.

B. NeuralNetclassifier

For the neuralnetclassifier, and the GridSearchCV over all model parameters, the data cost was the largest issue of this part of the project. The total data set was shortened into 1000 data points, 1000 data points out of a total 60 0000 training data. This could lead one to think that the network would not have enough data to become sufficiently accurate, but an accuracy of 89.22% for the final model on the test data shows that the model did learn, and classifies the unkown digits with a rather high accuracy. Adding a simple logistic regression model, and fitting the model on the same data, would have further shown how well the network actually performed, and this is a problem with this classification problem, it lacks models for comparison, as we had with the regression problem.

VI. CONCLUSION

Applied to the regression problem, the neural network showed a slightly lower test mse than the linear models, which is an impressive result. Although this is impressive, and shows that the neural network works, the data cost of the neural network, pushes the weight in favor of the linear models for solving this particular problem, the runge function on normal randomly distributed data.

When applying the same model structure to the classification problem, with the only significant difference being the output activation function, the cost function and the backpropagation. The model showed a great accuracy in predicting unknown digits, when trained on only 1000 digits. With code optimization or a computer with more power, the model could be trained on more data, and thus achieve a higher overall accuracy. A large portion of this project was used in interpreting the results of the GridSearchCV class created by Scikit learn, using different cross validation methods, and a better understanding of this class, would have further improved this project and the results of the training and cross validation. A large part of the success in this process can be attributed to stochastic gradient descent, the backpropagation alghoritm and the gradient updating method adam. Great methods for randomizing and increasing the training data, calculating gradients with a low data cost and for optimizing the gradients in an effective manner.

Appendix A: LLM Declaration

1. Tools used

- *[Tool/version; access method;* 22.10 − 9.11.2026*]*
- *GPT 6-Luna (codex),* 22.10 − 9.11.2026
- *[GPT 6-Luna/version; GitHub Copilot (coding assistant session);* 22.10 − 9.11.2026*]*

2. Text contributions

level 0 for no assistance. Level 1 for language-only edits Level 2 for editorial changes and structural drafting Level 3 for substantial drafting.

Abstract:: Level *[3]*; *[Copilot read through the rapport and the project, and wrote a summary which i verified and rewrote into the final abstract]*

Introduction:: Level *[1]*; *[Copilot created a basic motivation text which i used for inspiration, for the Motivation section.]*

Theory:: Level *[2]*; *[Copilot created the skeleton for equations, help with formatting and spacing of equations. Rewrote some equation descriptions]*

Backpropagation:: Level *[2]*; *[Copilot rewrote all equations describing backpropagation]*

Methods:: Level *[2]*; *[Copilot help with lists, figures and tables, formatting and structuring]*

Methods Subsections::

SGD:: Level *[3]*; *[ChatGPT-Sol 6.1 generated text describing the functions.]*

NNRegressor:: Level *[3]*; *[ChatGPT-Sol 6.1 generated text describing the functions.]*

NNClassifier:: Level *[3]*; *[ChatGPT-Sol 6.1 generated text describing the functions.]*

Results:: Level *[2]*; *[Copilot help with lists, figures and tables, formatting and structuring]*

Discussion:: Level *[0]*; *[Describe assistance, or write "none"]*

Conclusion:: Level *[0]*; *[Describe assistance, or write "none"]*

3. Code contributions

List each code file or notebook included in the submission. Use level 0 for no assistance, level 1 for debugging, level 2 for a snippet, level 3 for a skeleton, and level 4 for substantial generation. Include what was tested, adapted, or verified.

`[Project2/Strucutre*]`**::** Level *[3]*; *[Copilot imported all neccesary classes and functions for the python files to function]*

`[Project2/Code/Project2_Classification_Init.ipynb]`**::** Level *[2-3]*; *[Usage of classes objects. Translating functions into classes. Generated the one hot encoding of the target valeus, edited* `predict_proba` *and predict to function properly. Editorial changes and shape errors in training. Generate Heatmaps and Display of data using GridSearchCV. Usage of and retrieving data from GridsearchCV]*

`[Project2/Code/Project2_Regression_Init.ipynb]`**::** Level *[2-3]*; *[Usage pf classes and objects. Translating functions into classes. Debugging jax gradients and* `compute_gradients`*. Creating activation functions inside NeuralNetRegressor. Editorial changes and shape errors in training. Generate Heatmaps and Display of data using GridSearchCV. Usage of and retrieving data from GridsearchCV.]*

a. LLM Declaration this section:

The skeleton of this section was created using GitHub Copilot. (GPT Luna 6.0; coding-assistant session).

REFERENCES

(2026), "Confusion_matrix," https://scikit-learn/stable/modules/generated/sklearn.metrics.confusion_matrix.html.

(2026), "StratifiedKFold," https://scikit-learn/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html.

M. Hjorth-Jensen (2016), *Machine Learning and the Physical Sciences* (Springer).

M. Hjorth-Jensen (2026), "Project 2 on Machine Learning,".

P. Kaushyap (2024), "Understanding the Vanishing Gradient Problem in Deep Learning,".